Python Datatypes

List

- **List:** An ordered, mutable collection that allows duplicate elements.

  Example:

  ```
  my_list = [1, 2, 3, 'Python']
  ```

- **Creating List:** Lists are created using square brackets `[]`.

  Example:

  ```
  l = [10, 20, 30]
  ```

- **Accessing Items:** Elements are accessed using index numbers.

  Example:

  ```
  l = ['a', 'b', 'c']
  print(l[1])   # 'b'
  print(l[-1])  # 'c'
  ```

- **Slicing:** A range of elements can be accessed using slicing.

  Example:

  ```
  l = [10, 20, 30, 40]
  print(l[1:3])  # [20, 30]
  ```

- **Mutable:** Lists can be modified using index positions.

  Example:

  ```
  l = [1, 2, 3]
  l[1] = 10
  ```

- **Adding Items:**

- `append()` adds item to end.

```
l = [1, 2]
l.append(3)
```

- `insert()` adds item at index.

```
l.insert(1, 5)
```

- `extend()` adds multiple elements.

```
l.extend([6, 7])
```

- **Removing Items:**
  - `remove()` deletes value.

```
l.remove(5)
```

  - `pop()` removes by index.

```
l.pop(0)
```

  - `del` deletes item.

```
del l[0]
```

  - `clear()` empties list.

```
l.clear()
```

- **List Methods:**
  - `count()` counts occurrences.
  - `index()` finds position.
  - `reverse()` reverses list.
  - `sort()` sorts list.

- `copy()` copies list.

  Example:

```
l = [3, 1, 2]
l.sort()
l.reverse()
```

- **Sorting:**
  - `sort()` sorts original list.
  - `sorted()` returns new list.

    Example:

```
l = [3, 1, 2]
print(sorted(l))
```

- **Copying List:** Lists can be copied using `copy()` or slicing.

  Example:

```
l1 = [1, 2]
l2 = l1.copy()
```

- **Joining Lists:** Lists can be combined using `+` or `extend()`.

  Example:

```
l1 = [1, 2]
l2 = [3, 4]
print(l1 + l2)
```

Tuple

- **Tuple:** An ordered, immutable collection of items.

  Example:

```
my_tuple = (1, 2, 3, 'Python')
```

- **Creating Tuple:** Tuples are created using parentheses `()`.

  Example:

  ```
  t = (10, 20, 30)
  ```

- **Single Item Tuple:** Must include a comma.

  Example:

  ```
  t = (5,)
  ```

- **Accessing Items:** Elements are accessed using index numbers.

  Example:

  ```
  t = ('a', 'b', 'c')
  print(t[1])   # 'b'
  print(t[-1])  # 'c'
  ```

- **Slicing:** A range of elements can be accessed using slicing.

  Example:

  ```
  t = (10, 20, 30, 40)
  print(t[1:3])  # (20, 30)
  ```

- **Immutable:** Tuple values cannot be changed after creation.
- **Reassigning Tuple:** Entire tuple can be reassigned.

  Example:

  ```
  t = (1, 2, 3)
  t = (4, 5, 6)
  ```

- **Convert to List (Modify):** Convert to list to update values.

  Example:

  ```
  t = ('a', 'b', 'c')
  temp = list(t)
  ```

```
temp[1] = 'x'
t = tuple(temp)
```

- **Unpacking:** Assign tuple values to variables.

  Example:

```
t = ('apple', 'banana', 'cherry')
a, b, c = t
```

- *Unpacking with :* Store remaining values in a list.

  Example:

```
t = (1, 2, 3, 4, 5)
first, *mid, last = t
```

- **Joining Tuples:** Tuples can be combined using `+`.

  Example:

```
t1 = (1, 2)
t2 = (3, 4)
print(t1 + t2)
```

- **Tuple Methods:**
  - `count()` counts occurrences.
  - `index()` finds position.

    Example:

```
t = (1, 2, 2, 3)
print(t.count(2))   # 2
print(t.index(3))   # 3
```

- **Tuple vs List:** Tuples are immutable and faster, lists are mutable.
- **Deleting Tuple:** Entire tuple can be deleted using `del`.

  Example:

```
t = (1, 2, 3)
del t
```

Set

- **Set:** An unordered collection of unique elements.

  Example:

  ```
  s = {1, 2, 3}
  ```

- **Creating Set:** Sets are created using `{}` or `set()`.

  Example:

  ```
  s1 = {1, 2, 3}
  s2 = set([4, 5, 6])
  ```

- **Empty Set:** Created using `set()` not `{}`.

  Example:

  ```
  s = set()
  ```

- **No Indexing:** Sets do not support indexing (unordered).
- **Accessing Items:** Use loop or `in` keyword.

  Example:

  ```
  s = {1, 2, 3}
  print(2 in s)  # True
  ```

- **Adding Items:**
  - `add()` adds one item.

    ```
    s.add(4)
    ```

  - `update()` adds multiple items.

```
s.update([5, 6])
```

- **Removing Items:**
  - `remove()` deletes item (error if not found).

    ```
    s.remove(2)
    ```

  - `discard()` deletes item (no error).

    ```
    s.discard(10)
    ```

  - `pop()` removes random item.

    ```
    s.pop()
    ```

  - `clear()` empties set.

    ```
    s.clear()
    ```

- **Set Operations:**
  - `union()` combines sets.

    ```
    print({1,2} | {2,3})  # {1,2,3}
    ```

  - `intersection()` common elements.

    ```
    print({1,2,3} & {2,3,4})  # {2,3}
    ```

  - `difference()` elements in first not in second.

    ```
    print({1,2,3} - {2,3})  # {1}
    ```

  - `symmetric_difference()` elements not common.

    ```
    print({1,2,3} ^ {2,3,4})  # {1,4}
    ```

Dictionary

- **Dictionary:** An unordered collection of key-value pairs with unique keys.

  Example:

  ```
  d = {"name": "John", "age": 30}
  ```

- **Creating Dictionary:** Created using `{}` or `dict()`.

  Example:

  ```
  d1 = {"a": 1}
  d2 = dict(b=2)
  ```

- **Accessing Items:** Values are accessed using keys.

  Example:

  ```
  print(d["name"])
  print(d.get("age"))
  ```

- **Mutable:** Dictionary values can be changed or added.

  Example:

  ```
  d["age"] = 31
  d["city"] = "NY"
  ```

- **Removing Items:**
  - `pop()` removes by key.

    ```
    d.pop("age")
    ```

  - `popitem()` removes last item.

    ```
    d.popitem()
    ```

  - `del` deletes key.

```
del d["name"]
```

- `clear()` empties dictionary.

```
d.clear()
```

- **Copy Dictionary:** Creates a copy.

  Example:

```
d2 = d.copy()
```

- **Nested Dictionary:** Dictionary inside another dictionary.

  Example:

```
d = {
    "p1": {"name": "John", "age": 30},
    "p2": {"name": "Alice", "age": 25}
}
print(d["p1"]["name"])
```

- **Dictionary Methods:**
  - `keys()` returns keys.
  - `values()` returns values.
  - `items()` returns key-value pairs.

    Example:

```
print(d.keys())
print(d.values())
print(d.items())
```

- **update():** Updates dictionary with new key-value pairs.

  Example:

```
d.update({"city": "NY"})
```